

Data Cleaning and Optimization

In this notebook, we will perform data cleaning and optimization on the diamonds dataset. This dataset contains information on diamonds such as their shape, weight, clarity, colour, cut, polish, symmetry, fluorescence, measurements and price. The goal of this notebook is to explore the data, handle missing values, duplicates, and clean the dataset to prepare it for analysis.

Importing Libraries

We will start by importing the necessary libraries that we will be using throughout the notebook.

- `pandas` - for data manipulation and analysis
- `numpy` - for numerical operations
- `matplotlib` - for data visualization
- `seaborn` - for advanced data visualization

```
In [ ]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
```

Loading Data

Next, we will load the dataset using `pandas read_csv` function.

```
In [ ]: df = pd.read_csv("../data/diamonds.csv")
df
```

Out []:

	Id	Shape	Weight	Clarity	Colour	Cut	Polish	Symmetry	Fluorescence	Messurements	Price
	0	1638147	CUSHION	0.55	SI2	E	EX	EX	VG	N 5.05-4.35×2.94	1,378.65
	1	1630155	CUSHION	0.50	VVS1	FANCY	EX	EX	VG	F 4.60-4.31×2.92	1,379.74
	2	1612606	CUSHION	0.51	VS2	H	EX	EX	VG	N 4.71-4.35×2.94	1,380.19
	3	1638140	CUSHION	0.50	VS2	H	EX	EX	VG	N 4.91-4.26×2.88	1,380.61
	4	1536093	CUSHION	0.53	SI1	D	EX	VG	VG	N 4.70-4.46×3.01	1,383.13
	...	...	...	...	...	...	...	...	...	...	...
6334	1751206	ROUND	0.50	SI1	E	EX	EX	EX	N	5.00-5.04×3.18	2,453.10
6335	1773024	ROUND	0.50	SI1	E	EX	EX	EX	N	5.03-5.06×3.16	2,453.10
6336	1789435	ROUND	0.50	VS2	D	VG	EX	VG	N	4.93-4.96×3.19	2,453.20
6337	1774887	ROUND	0.52	SI1	H	EX	EX	EX	N	5.11-5.13×3.20	2,453.41
6338	1630512	ROUND	0.50	VS1	H	EX	EX	VG	N	5.09-5.13×3.12	2,453.69

6339 rows × 11 columns

Data Exploration

Let's explore the data by checking the columns and their data types.

```
In [ ]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6339 entries, 0 to 6338
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Id           6339 non-null   object
1   Shape        6339 non-null   object
2   Weight       6339 non-null   float64
3   Clarity      6319 non-null   object
4   Colour       6339 non-null   object
5   Cut          6337 non-null   object
6   Polish       6338 non-null   object
7   Symmetry     6332 non-null   object
8   Fluorescence 6337 non-null   object
9   Messurements 6339 non-null   object
10  Price        6339 non-null   object
dtypes: float64(1), object(10)
memory usage: 544.9+ KB
```

Handling Missing Values

We can also check for missing values and visualize them using a bar plot and a heatmap.

```
In [ ]: missing_values = df.isnull().sum()
missing_values
```

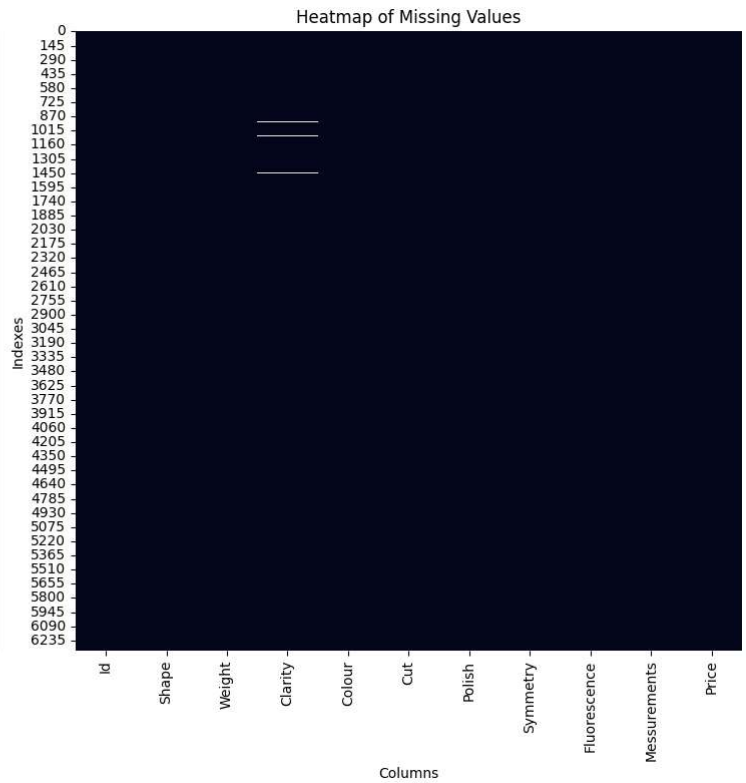
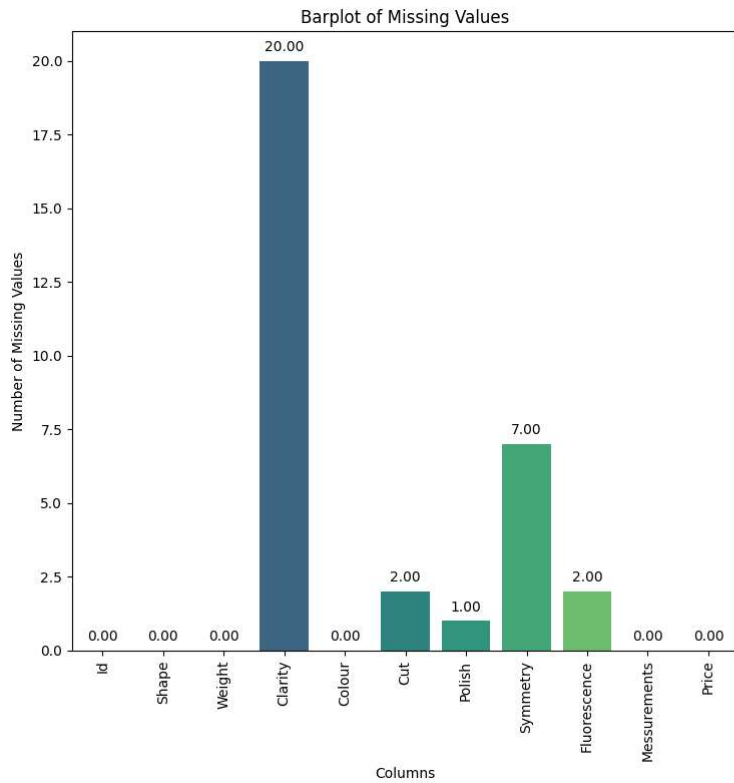
```
Out [ ]: Id           0
Shape          0
Weight         0
Clarity        20
Colour         0
Cut            2
Polish         1
Symmetry       7
Fluorescence   2
Messurements   0
Price          0
dtype: int64
```

```
In [ ]: fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(15, 8))

sns.barplot(x=missing_values.index, y=missing_values.values, palette="viridis", ax=ax1)
ax1.set_title("Barplot of Missing Values")
ax1.set_xticklabels(ax1.get_xticklabels(), rotation=90)
ax1.set_xlabel("Columns")
ax1.set_ylabel("Number of Missing Values")
for p in ax1.patches:
    ax1.annotate(
        format(p.get_height(), ".2f"),
        (p.get_x() + p.get_width() / 2.0, p.get_height()),
        ha="center",
        va="center",
        xytext=(0, 10),
        textcoords="offset points",
    )
```

```
sns.heatmap(df.isnull(), cbar=False, ax=ax2)
ax2.set_title("Heatmap of Missing Values")
ax2.set_xlabel("Columns")
ax2.set_ylabel("Indexes")

plt.tight_layout()
plt.show()
```



We will fill the missing values in the dataset with the mode value of the respective columns.

```
In [ ]: df["Clarity"].fillna(df["Clarity"].mode()[0], inplace=True)
df["Cut"].fillna(df["Cut"].mode()[0], inplace=True)
df["Polish"].fillna(df["Polish"].mode()[0], inplace=True)
df["Symmetry"].fillna(df["Symmetry"].mode()[0], inplace=True)
df["Fluorescence"].fillna(df["Fluorescence"].mode()[0], inplace=True)
```

Check wether we solve missing values

```
In [ ]: df.isnull().sum()
```

```
Out [ ]: Id      0
Shape    0
Weight   0
Clarity  0
Colour   0
Cut      0
Polish   0
Symmetry 0
Fluorescence 0
Measurements 0
Price    0
dtype: int64
```

Handling Duplicates

We will check for any duplicate values in the dataset and visualize them using a bar plot and a pie chart.

```
In [ ]: num_duplicates = df.duplicated().sum()
print(f"Number of duplicates values present in dataframe: {num_duplicates}")

Number of duplicates values present in dataframe: 348
```

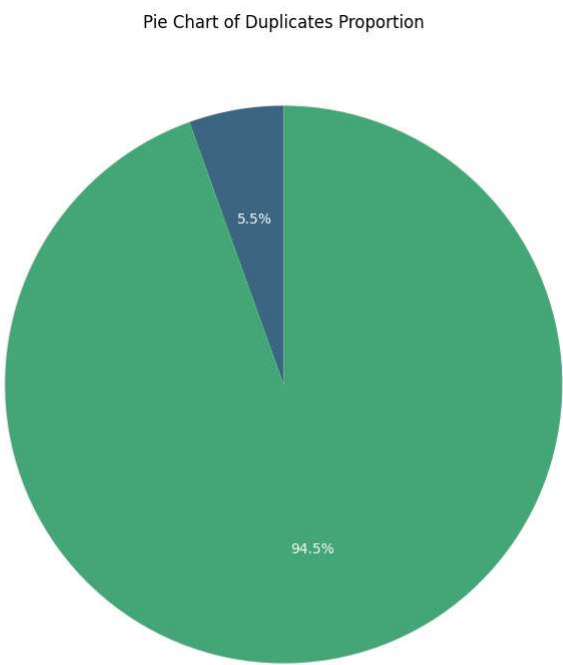
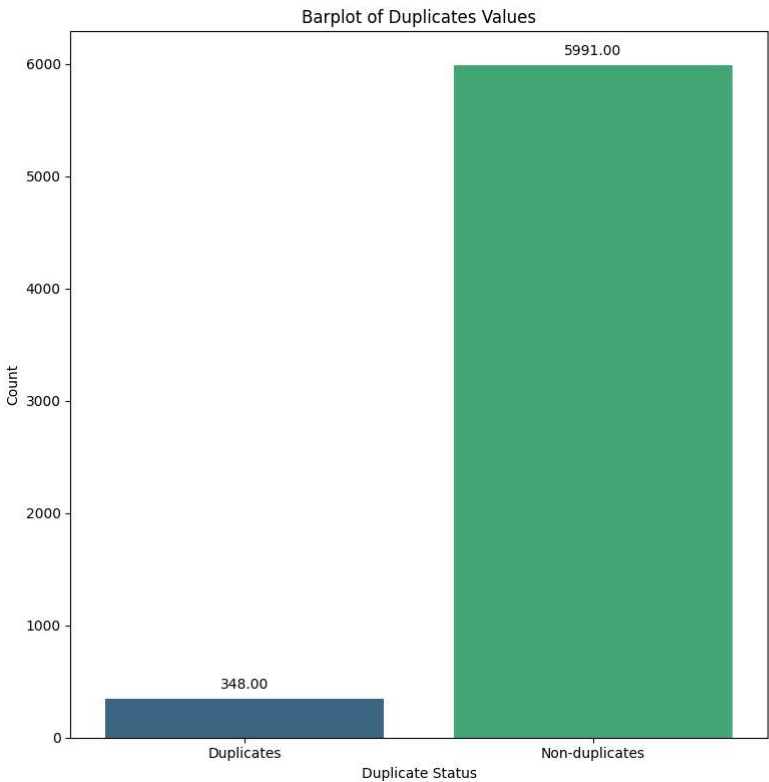
```
In [ ]: fig, (ax1, ax2) = plt.subplots(ncols=2, figsize=(15, 8))
```

```
sns.barplot(
    x=["Duplicates", "Non-duplicates"],
    y=[num_duplicates, len(df) - num_duplicates],
    palette="viridis",
    ax=ax1,
)
ax1.set_title("Barplot of Duplicates Values")
ax1.set_xlabel("Duplicate Status")
ax1.set_ylabel("Count")
for p in ax1.patches:
    ax1.annotate(
        format(p.get_height(), ".2f"),
        (p.get_x() + p.get_width() / 2.0, p.get_height()),
        ha="center",
        va="center",
        xytext=(0, 10),
        textcoords="offset points",
    )

labels = ["Duplicates", "Non-duplicates"]
sizes = [num_duplicates, len(df) - num_duplicates]
ax2.pie(
    sizes,
    labels=labels,
    autopct="%1.1f%%",
    startangle=90,
    colors=["#3c6682", "#45a778"],
    textprops={"color": "white"},
)
```

```
ax2.set_title("Pie Chart of Duplicates Proportion")

plt.tight_layout()
plt.show()
```



We will then drop the duplicate values in the dataset.

```
In [ ]: df = df.drop_duplicates()

Check wether we solve duplicate values

In [ ]: print(f"Number of duplicates values present in dataframe: {df.duplicated().sum()}")

Number of duplicates values present in dataframe: 0
```

Data Cleaning 🧹

Checking the columns and their data types

```
In [ ]: df.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 5991 entries, 0 to 6338
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0    Id          5991 non-null   object
1    Shape       5991 non-null   object
2    Weight      5991 non-null   float64
3    Clarity     5991 non-null   object
4    Colour      5991 non-null   object
5    Cut         5991 non-null   object
6    Polish     5991 non-null   object
7    Symmetry    5991 non-null   object
8    Fluorescence 5991 non-null   object
9    Measurements 5991 non-null   object
10   Price       5991 non-null   object
dtypes: float64(1), object(10)
memory usage: 561.7+ KB
```

Check the format of Price

```
In [ ]: df.Price

Out[ ]: 0      1,378.65
1      1,379.74
2      1,380.19
3      1,380.61
4      1,383.13
...
6332   2,453.10
6333   2,453.10
6336   2,453.20
6337   2,453.41
6338   2,453.69
Name: Price, Length: 5991, dtype: object
```

Insight 💡

The values in the 'Price' column are strings that represent numbers with commas as thousands separators.

So to convert into float replace ',' to ''

```
In [ ]: df["Price"] = df["Price"].str.replace(",","").astype(float)
```

Check the format of Measurements

```
In [ ]: df.Measurements
```

```
Out [ ]: 0      5.05-4.35x2.94
1      4.60-4.31x2.92
2      4.71-4.35x2.94
3      4.91-4.26x2.88
4      4.70-4.46x3.01
...
6332   5.00-5.04x3.18
6333   5.03-5.06x3.16
6336   4.93-4.96x3.19
6337   5.11-5.13x3.20
6338   5.09-5.13x3.12
Name: Measurements, Length: 5991, dtype: object
```

Insight💡

The values in the Measurements column are strings that represent Length, Width & Depth of the diamond separated by - & x in a format of Length-WidthxDepth

So Measurements column is split into Length, Width & Depth columns

Finally, drop Measurements column because it has no use

```
In [ ]: def getNumbers(text: str) -> list:
import re

return re.findall(r"[0-9]", text)

df["Measurements"] = df["Measurements"].apply(lambda x: getNumbers(x))
df["Measurements"] = df["Measurements"].apply(lambda x: "".join(x))

df["Length"] = df["Measurements"].str[:3].astype(float) / 100
df["Width"] = df["Measurements"].str[3:6].astype(float) / 100
df["Depth"] = df["Measurements"].str[6:].astype(float) / 100

df.drop(["Measurements"], axis=1, inplace=True)
```

Check the dataframe

```
In [ ]: df
```

```
Out [ ]:
```

		Id	Shape	Weight	Clarity	Colour	Cut	Polish	Symmetry	Fluorescence	Price	Length	Width	Depth
	0	1638147	CUSHION	0.55	SI2	E	EX	EX	VG	N	1378.65	5.05	4.35	2.94
	1	1630155	CUSHION	0.50	VVS1	FANCY	EX	EX	VG	F	1379.74	4.60	4.31	2.92
	2	1612606	CUSHION	0.51	VS2	H	EX	EX	VG	N	1380.19	4.71	4.35	2.94
	3	1638140	CUSHION	0.50	VS2	H	EX	EX	VG	N	1380.61	4.91	4.26	2.88
	4	1536093	CUSHION	0.53	SI1	D	EX	VG	VG	N	1383.13	4.70	4.46	3.01
	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	6332	1751206	ROUND	0.50	SI1	E	EX	EX	EX	N	2453.10	5.00	5.04	3.18
	6333	1773024	ROUND	0.50	SI1	E	EX	EX	EX	N	2453.10	5.03	5.06	3.16
	6336	1789435	ROUND	0.50	VS2	D	VG	EX	VG	N	2453.20	4.93	4.96	3.19
	6337	1774887	ROUND	0.52	SI1	H	EX	EX	EX	N	2453.41	5.11	5.13	3.20
	6338	1630512	ROUND	0.50	VS1	H	EX	EX	VG	N	2453.69	5.09	5.13	3.12

5991 rows x 13 columns

Checking the unique values in Cut, Polish and Symmetry

```
In [ ]: df.Cut.unique()

Out [ ]: array(['EX', 'VG', 'GD', 'F'], dtype=object)

In [ ]: df.Polish.unique()

Out [ ]: array(['EX', 'VG', 'GD', 'F'], dtype=object)

In [ ]: df.Symmetry.unique()

Out [ ]: array(['VG', 'EX', 'GD', 'FR'], dtype=object)

In [ ]: df["Cut"] = df["Cut"].replace("F", "FR")
df["Polish"] = df["Polish"].replace("F", "FR")
```

Checking the unique values in Cut, Polish and Symmetry after replace

```
In [ ]: df.Cut.unique()

Out [ ]: array(['EX', 'VG', 'GD', 'FR'], dtype=object)

In [ ]: df.Polish.unique()

Out [ ]: array(['EX', 'VG', 'GD', 'FR'], dtype=object)
```

Insight💡

We found that Cut, Polish and Symmetry have similar values

So we will rename F to FR in Cut and Polish for uniformity

Checking the columns and their data types

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 5991 entries, 0 to 6338
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  -
0    Id           5991 non-null   object
1    Shape        5991 non-null   object
2    Weight       5991 non-null   float64
3    Clarity      5991 non-null   object
4    Colour       5991 non-null   object
5    Cut          5991 non-null   object
6    Polish       5991 non-null   object
7    Symmetry     5991 non-null   object
8    Fluorescence 5991 non-null   object
9    Price        5991 non-null   float64
10   Length       5991 non-null   float64
11   Width        5991 non-null   float64
12   Depth        5991 non-null   float64
dtypes: float64(5), object(8)
memory usage: 655.3+ KB
```

Optimize dataframe

Optimizes a dataframe by reducing its memory usage

```
In [ ]: from typing import Any
```

```
def optimize_dataframe(df: pd.DataFrame) -> pd.DataFrame:
    optimized_df = df.copy()

    # Iterate through each column
    for col in optimized_df.columns:
        col_data = optimized_df[col]
        dtype = col_data.dtype

        # Convert object columns to category dtype if less than 50% unique
        if dtype == object:
            _object_optimizer(optimized_df, col, col_data)

        elif dtype == "int64":
            _int_optimizer(optimized_df, col, col_data)

        elif dtype == "float64":
            _float_optimizer(optimized_df, col, col_data)

    # Print memory usage before and after optimization
    before_mem = df.memory_usage().sum() / 1024**2
    after_mem = optimized_df.memory_usage().sum() / 1024**2
    print(f"Memory usage before optimization: {before_mem:.2f} MB")
    print(f"Memory usage after optimization: {after_mem:.2f} MB")

    return optimized_df

def _object_optimizer(optimized_df: pd.DataFrame, col: Any, col_data: Any) -> None:
    num_unique = len(col_data.unique())
    num_total = len(col_data)
    if num_unique / num_total < 0.5:
        optimized_df[col] = col_data.astype("category")

def _int_optimizer(optimized_df: pd.DataFrame, col: Any, col_data: Any) -> None:
    col_min = col_data.min()
    col_max = col_data.max()
    if col_min >= 0:
        if col_max < 256:
            optimized_df[col] = col_data.astype("uint8")
        elif col_max < 65536:
            optimized_df[col] = col_data.astype("uint16")
        elif col_max < 4294967296:
            optimized_df[col] = col_data.astype("uint32")
        else:
            optimized_df[col] = col_data.astype("uint64")
    elif col_min > np.iinfo(np.int8).min and col_max < np.iinfo(np.int8).max:
        optimized_df[col] = col_data.astype("int8")
    elif col_min > np.iinfo(np.int16).min and col_max < np.iinfo(np.int16).max:
        optimized_df[col] = col_data.astype("int16")
    elif col_min > np.iinfo(np.int32).min and col_max < np.iinfo(np.int32).max:
        optimized_df[col] = col_data.astype("int32")
    else:
        optimized_df[col] = col_data.astype("int64")

def _float_optimizer(optimized_df: pd.DataFrame, col: Any, col_data: Any) -> None:
    col_min = col_data.min()
    col_max = col_data.max()
    if col_min >= np.finfo(np.float16).min and col_max < np.finfo(np.float16).max:
        optimized_df[col] = col_data.astype("float16")
    elif col_min >= np.finfo(np.float32).min and col_max < np.finfo(np.float32).max:
        optimized_df[col] = col_data.astype("float32")
    else:
        optimized_df[col] = col_data.astype("float64")
```

Calling `optimize_dataframe` function to oue dataframe

```
In [ ]: optimized_df = optimize_dataframe(df)

Memory usage before optimization: 0.64 MB
Memory usage after optimization: 0.20 MB

Final checking the columns and their data types
```

```
In [ ]: optimized_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 5991 entries, 0 to 6338
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     5991 non-null   object
1   Shape                  5991 non-null   category
2   Weight                 5991 non-null   float16
3   Clarity                5991 non-null   category
4   Colour                 5991 non-null   category
5   Cut                    5991 non-null   category
6   Polish                 5991 non-null   category
7   Symmetry               5991 non-null   category
8   Fluorescence           5991 non-null   category
9   Price                  5991 non-null   float32
10  Length                 5991 non-null   float16
11  Width                  5991 non-null   float16
12  Depth                  5991 non-null   float16
dtypes: category(7), float16(4), float32(1), object(1)
memory usage: 207.1+ KB
```

Saving the optimized dataframe

Save the dataframe in `../data/clean_diamonds.csv` without index

```
In [ ]: optimized_df.to_csv("../data/clean_diamonds.csv", index=False)
```

Also saving in parquet file using BROTLI compression and this will be save at `../data/clean_diamonds.br`

```
In [ ]: optimized_df.to_parquet("../data/clean_diamonds.br", compression="BROTLI")
```

Conclusion

In this notebook, we performed data cleaning and optimization on the diamonds dataset. We started by importing necessary libraries, loading the dataset, and exploring the data by checking the columns and their data types. We then checked for missing values and handled them by filling them with the mode value of the respective columns. Next, we checked for duplicate values and dropped them from the dataset.

We also performed data cleaning by converting the 'Price' column from strings to float and splitting the 'Measurements' column into 'Length', 'Width', and 'Depth' columns and also dropped the 'Measurements' column as it was no longer required.

Finally, we also apply optimization by reducing its memory usage and saved it in csv as well as parquet format

Overall, these steps helped us to clean and optimize the dataset, making it ready for further analysis and modeling.